



# Come prevenire il degrado architettonale\_

— Mengoli Alessandro

















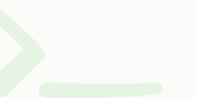






Il degrado non è una decisione.  
È un' **accumulo**.

Invisibile, finché non diventa costoso.

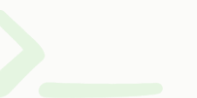




# bit rot\_

Il deterioramento graduale della qualità del software nel tempo

Il bit rot non erode "il codice". Erode delle **caratteristiche**.





# Ogni sintomo è una caratteristica che si erode

---

"non toccare, è tutto mischiato"

→ modularità

---

paura del deploy, rilasci rari

→ deployability

---

"ogni fix ne rompe due"

→ testability

---

il Controller che chiama il DB

→ modularità

---

stime che esplodono nel tempo

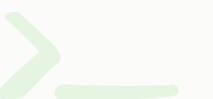
→ manutenibilità / evolvibilità

---

dipendenze troppo vecchie e vulnerabili

→ manutenibilità / sicurezza

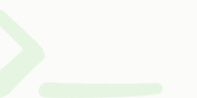
---





## Alcune caratteristiche (-ilities)

modularity **testability** deployability scalability performance  
**security** reliability observability **maintainability** evolvability  
portability interoperability

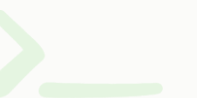








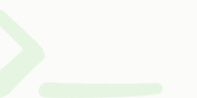
Degrado = erosione di caratteristiche.  
Quindi prevenire = proteggerele.





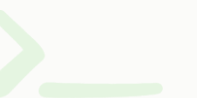
# Non puoi proteggerle **tutte**.

Sono in trade-off tra loro: massimizzarne una ne sacrifica un'altra.





Una caratteristica che non sai misurare è solo un  
desiderio.





# Dall'aggettivo alla soglia

---

"deve essere scalabile"

→ `p99 < 200ms a 10.000 req/s`

---

"deve essere modulare"

→ `Domain: zero deps su Infrastructure`

---

"deve essere deployabile"

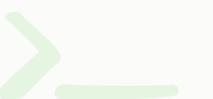
→ `lead time commit → prod < 15 min`

---

"deve essere sicuro"

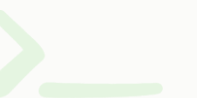
→ `zero pacchetti con CVE note in prod`

---



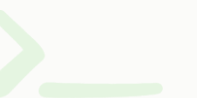


Rendi una -ility oggettiva ed eseguibile, e hai costruito una  
fitness function.





"A fitness function provides an **objective** integrity assessment of some architectural characteristic."





oggettiva

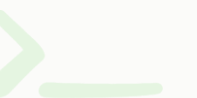
un numero, un pass/fail — non un'opinione

integrità

è ancora dentro i limiti che volevamo?

caratteristica

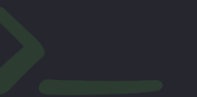
la -ility







# Demo







Degrada relativamente a un mondo che si muove:  
requisiti, dipendenze, runtime, team.



Il software è vivo. **Evolve.**





Il software è vivo. **Evolve.**

L'architettura non si decide una volta sola. **Evolve.**



"An evolutionary architecture supports **guided,**  
**incremental** change across multiple dimensions."

Ford, Parsons, Kua - Building Evolutionary Architectures





"An evolutionary architecture supports **guided,**  
**incremental** change across multiple dimensions."

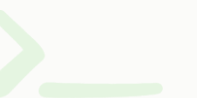
Ford, Parsons, Kua - Building Evolutionary Architectures







Regole architetturali scritte come **test** , eseguite dal runner che già usi.  
Fanno fallire la CI e bloccano la PR che erode l'architettura.



Ma non solo...



# I cinque assi delle fitness function

Scope

Atomic ↔ Holistic

Cadence

Triggered ↔ Continual ↔ Temporal

Result

Static ↔ Dynamic

Invocation

Automated ↔ Manual

Proactivity

Intentional ↔ Emergent





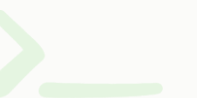
# Gli assi, in sintesi

| ASSE        | LA DOMANDA                     | ESEMPIO .NET                     |
|-------------|--------------------------------|----------------------------------|
| Scope       | un aspetto o una combinazione? | ArchUnitNet vs "no PII nei log"  |
| Cadence     | quando viene valutata?         | dotnet test vs monitor p99       |
| Result      | soglia fissa o contestuale?    | coverage ≥ 80% vs latency budget |
| Invocation  | umano nel loop?                | gate in CI vs ADR review         |
| Proactivity | pianificata o scoperta?        | layering vs limite sotto carico  |





Una fitness function **non è** sinonimo di test xUnit.





# Strumenti per tipologia

|                 |                              |
|-----------------|------------------------------|
| Static analysis | Roslyn · SonarQube · NDepend |
|-----------------|------------------------------|

|          |                          |
|----------|--------------------------|
| Security | CVE scan · CodeQL · Snyk |
|----------|--------------------------|

|                  |         |
|------------------|---------|
| Contract testing | PactNet |
|------------------|---------|

|                     |                    |
|---------------------|--------------------|
| Observability / SLO | alert p99 · canary |
|---------------------|--------------------|

|                  |             |
|------------------|-------------|
| Mutation testing | Stryker.NET |
|------------------|-------------|

|             |                               |
|-------------|-------------------------------|
| Performance | NBomber · k6 · latency budget |
|-------------|-------------------------------|

|                   |               |
|-------------------|---------------|
| Chaos engineering | Polly · Simmy |
|-------------------|---------------|

|                  |                        |
|------------------|------------------------|
| Policy · Manuali | OPA · ADR review board |
|------------------|------------------------|

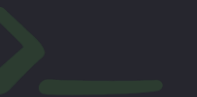


# Strumenti per caratteristiche

| ESEMPIO            | SCOPE    | CADENCE   | RESULT  |
|--------------------|----------|-----------|---------|
| Roslyn analyzer    | Atomic   | Triggered | Static  |
| ArchUnitNet        | Atomic   | Triggered | Static  |
| NBomber latency    | Atomic   | Triggered | Dynamic |
| SLO p99 in prod    | Holistic | Continual | Dynamic |
| ADR / review board | Holistic | Triggered | Manual  |



# Demo





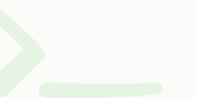
# Tutto insieme, in pipeline

61

```
- name: Quality & architecture gates  run: |  
  dotnet format --verify-no-changes  
  dotnet test --filter "Category=Architecture"  
  dotnet test --filter "Category=Contract"  
  dotnet list package --vulnerable --include-transitive  
  dotnet stryker --break-at 60
```

La fitness function è un **concetto**, non una libreria.

Compilatore, test, metriche, monitoring, policy, chaos, ecc.

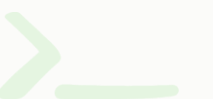






# Takeaways

- 01 Scegli le 3-5 caratteristiche da proteggere
- 02 Rendile misurabili, ogni aggettivo diventa una soglia
- 03 Aggiungi una fitness function (facile con ArchUnitNet)
- 04 Rendi le regressioni impossibili da ignorare, una fitness function alla volta
- 05 Rivedile periodicamente — la suite è viva
- 06 L'architettura è un lavoro di team





# About&Resources



`amengoli9/talks/`



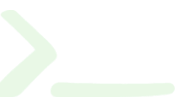
`dev-pills.net`



`devromagna.org`



Alessandro Mengoli  
Technical Leader @ SACMI





Come evitare il degrado architettuale

